



Security Assessment & Formal Verification Draft Report



Iota Coin

March 2026

Prepared for Iota Foundation

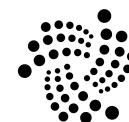


Table of Contents

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Threat and Security Overview.....	4
Findings Summary.....	5
Severity Matrix.....	5
Detailed Findings.....	6
Informational Issues.....	7
I-01. Dust Incorrectly Forwarded to Fee Recipient.....	7
I-02. quote_of() under-reports the true maximum sendable amount when outbound fees are enabled.....	8
I-03. Simple receives do not reject zero address, allowing irreversible loss of bridged funds.....	9
Formal Verification.....	10
Verification Methodology.....	10
Verification Notations.....	11
General Assumptions and Simplifications.....	11
Formal Verification Properties Overview.....	12
Detailed Properties.....	12
Coin.....	13
Disclaimer.....	19
About Certora.....	19



Project Summary

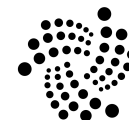
Project Scope

Project Name	Repository (link)	First Commit Hash	Latest Commit Hash	Platform
Iota / Coin	https://github.com/iotaledger/iota/blob/develop/crates/iota-framework/packages/iota-framework/sources/coin.move	3f1a491		Move
USDTO	https://github.com/LayerZero-Labs/usdt0-iota-contracts	c213287		Move

Project Overview

USDTO is an omnichain stablecoin by Tether, deployed on IOTA using the Move programming language and integrated with LayerZero's OFT standard. It follows a mint-and-burn model.

The token module uses IOTA's regulated currency primitive to produce a **TreasuryCap<USDTO>** (mint/burn authority) and a **DenyCap<USDTO>** (pause and denylist authority). A One-Time Witness ensures the **TreasuryCap** is created exactly once. At OFT initialization, it is permanently consumed into the shared **OFT<USDTO>** object, making the OFT bridge the sole authority over token supply on IOTA. The OFT layer then acts as the bridge integration point, handling cross-chain sends, receives, administrative configuration, and token supply movements on IOTA.



Threat and Security Overview

This system follows a bridge-centric mint-and-burn model in which USDT0 is issued on IOTA as a regulated Move coin and integrated with LayerZero's OFT framework for cross-chain transfers.

The core security boundary is the custody and use of two token authorities:

`TreasuryCap<USDT0>` governs mint and burn, while `DenyCap<USDT0>` governs denylist and pause behavior. Once initialized, the treasury authority is embedded into the shared `OFT<USDT0>` object, making the OFT layer the effective supply controller on IOTA. The main expected invariants are that supply changes only through intended bridge flows or explicitly authorized admin paths, cross-chain send and receive flows remain internally consistent, and regulated-token controls apply correctly to bridged transfers.

The two primary trust boundaries are privileged roles and external bridge dependencies. The OFT admin surface controls operational settings such as pause state, fees, rate limits, and metadata, while also exposing direct supply-mutating paths, `mint()` and `redeem()`, that bypass bridge flows entirely. LayerZero message delivery, OApp peer configuration, and offchain DVN-based controls form a second boundary outside the token logic itself. Formal verification focused on `usdt0.move` and the underlying coin module, covering supply integrity, initialization correctness, balance operation behavior, denylist and pause mechanics, and metadata update integrity. Manual review covered the token initialization flow, the treasury and denylist authority model including the one-time transfer of `TreasuryCap` into the OFT, OFT send and receive paths, admin-only functions, migration and metadata surfaces, and quote and rate-limit logic.

Overall, the security posture of USDT0 on IOTA appears sound. Several OFT-layer issues were identified and are included in the report for completeness, but none materially affect USDT0 itself as rate limiting is enforced offchain via LayerZero DVNs, decimal handling is symmetric across chains, and certain behaviors are intentional OFT design choices. The coin module invariants were formally verified to hold.



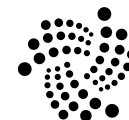
Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	0	-	-
High	0	-	-
Medium	0	-	-
Low	0	-	-
Informational	3	-	-
Total	3		

Severity Matrix

Impact		Likelihood		
		Low	Medium	High
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium



Detailed Findings

ID	Title	Severity	Status
I-01	Dust Incorrectly Forwarded to Fee Recipient	Informational	
I-02	quote_offt() under-reports the true maximum sendable amount when outbound fees are enabled	Informational	
I-03	Simple receives do not reject zero address, allowing irreversible loss of bridged funds	Informational	

Informational Issues

I-01. Dust Incorrectly Forwarded to Fee Recipient

Description:

`debit()` splits the user's coin using the gap between `amount_sent_ld` and `amount_received_ld` and forwards it to `fee_deposit_address`. Whatever lands in that gap goes to the fee recipient.

In the no-fee path, `amount_sent_ld = remove_dust(amount_ld)`, so both values are equal, the gap is zero, and dust remains with the user. In the fee path, however, `debit_view_with_fee()` returns raw `amount_ld` as `amount_sent_ld` instead of a dust-aligned value, so the gap includes both the intended fee and any dust remainder:

None

```
(amount_ld, amount_received_ld) // amount_sent_ld = raw input, not
dust-aligned
```

As a result, dust is forwarded to `fee_deposit_address` instead of remaining in the user's coin.

This is not applicable in the current USDT0 deployment because decimals are symmetric across chains, so `decimal_conversion_rate = 1` and no dust is produced in practice.

Recommendation:

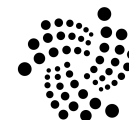
None

```
let fee = amount_ld - amount_ld_after_fee;
let amount_sent_ld = amount_received_ld + fee; // dust excluded
```

The gap becomes exactly the fee. Dust stays with the user, consistent with the no-fee path.

Customer's response: Awaiting customer response.

Fix Review:



I-02. `quote_offt()` under-reports the true maximum sendable amount when outbound fees are enabled

Description:

`quote_offt()` is intended to estimate fees and operational transfer limits. It currently returns the available capacity of the `outbound_rate_limiter` directly as `max_amount_ld`. However, the send path consumes rate-limit capacity based on `amount_received_ld`, which is the net amount after outbound fees are deducted, not the gross user input.

As a result, `quote_offt()` under-reports the true maximum gross amount a user can send whenever outbound fees are enabled. Funds remain safe, but the quote is unnecessarily conservative and may cause frontends to restrict usable bridge capacity.

This issue is not applicable in the current USDT0 setting because rate limiting is handled offchain via LayerZero DVNs rather than relying on this onchain OFT limiter logic.

Recommendation:

Adjust the `max_amount_ld` calculation in `quote_offt()` to account for outbound fees. It should reverse-calculate the maximum gross input such that the post-fee `amount_received_ld` equals the available rate-limit capacity.

Customer's response: Awaiting customer response.

Fix Review:

I-03. Simple receives do not reject zero address, allowing irreversible loss of bridged funds

Description:

The bridge accepts a raw **Bytes32** destination, and the simple receive path forwards the credited coin directly to **send_to** without rejecting zero address. A user can inadvertently bridge tokens to the zero address on the source chain, and the destination side will successfully transfer the resulting coin to zero address, where it becomes permanently unrecoverable in IOTA Move.

Recommendation:

Reject zero address, explicitly in the source send path for IOTA destinations, and enforce this again in the destination receive path before delivering funds, to prevent irreversible user error.

Customer's response: Awaiting customer response.

Fix Review:

Formal Verification

Verification Methodology

We performed verification of the USDTO contracts on IOTA, which are deployed using LayerZero's OFT standard, using the Certora verification tool, which is based on Satisfiability Modulo Theories (SMT). In short, the Certora verification tool works by compiling formal specifications written in the [Certora Verification Language for Move \(CVLM\)](#) and Iota's implementation source code written in Move. More information about Certora's tooling can be found in the [Certora Technology Whitepaper](#).

If a property is verified with this methodology it means the specification in CVLM holds for all possible inputs. However specifications must introduce assumptions to rule out situations which are impossible in realistic scenarios (e.g. to specify the valid range for an input parameter). Additionally, SMT-based verification is notoriously computationally difficult. As a result, we introduce overapproximations (replacing real computations with broader ranges of values) and underapproximations (replacing real computations with fewer values) to make verification feasible.

Rules: A rule is a verification task possibly containing assumptions, calls to the relevant functionality that is symbolically executed and assertions that are verified on any resulting states from the computation.



Verification Notations

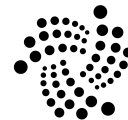
Formally Verified	The rule is verified for every state of the contract(s), under the assumptions of the scope/requirements in the rule.
Formally Verified After Fix	The rule was violated due to an issue in the code and was successfully verified after fixing the issue
Violated	A counter-example exists that violates one of the assertions of the rule.

General Assumptions and Simplifications

- By default, all loops are unrolled at most twice.
- The move runtime environment is modeled by the Certora Prover.
- Parametric rules are checked against all public functions in the coin module.

Formal Verification Properties Overview

ID	Title	Impact	Status
P-01	TotalSupplyUnchanged	Total supply increases only in mint, decreases only in burn. The sum of balances is equal to the total supply.	Verified
P-02	TotalSupply	Total supply is modified correctly in mints and burns.	Verified
P-03	Balances	Balance and coin operations behave as expected.	Verified
P-04	Initialization	Currency creation establishes the correct initial state.	Verified
P-05	DenyList	Deny List and global pause operations behave as expected.	Verified
P-06	Metadata	Metadata update functions update as expected.	Verified



Detailed Properties

Coin

P-01. TotalSupplyUnchanged

Status: Verified

Total Supply increases only by mint, mint_balance, mint_and_transfer and decreases only by burn, total supply is equal to the sum of balances.

Rule Name	Status	Description	Link to rule report
only_mint_can_increase_total_supply	Verified	<i>only mint, mint_balance and mint_and_transfer can increase total supply</i>	rule report
only_burn_can_decrease_total_supply	Verified	<i>only burn can decrease total supply</i>	rule report
total_supply_equal_sum_of_balances	Verified	<i>total supply should be equal to sum of balances</i>	rule report



P-02. TotalSupply

Status: Verified

Total supply is updated correctly by mint, mint_balance, and burn.

Rule Name	Status	Description	Link to rule report
mint_total_supply	Verified	<i>after mint supply is increased by the value of the minted coin</i>	rule report
mint_balance_total_supply	Verified	<i>after mint_balance is called, total supply is increased by the value of the minted coin</i>	rule report
burn_total_supply	Verified	<i>after burn is called, total supply is decreased by the value of the burned coin</i>	rule report

P-03. Balances

Status: Verified

Balance and coin operations preserve or update values as specified (take, put, join, split, divide_into_n, mint, burn, zero, etc.).

Rule Name	Status	Description	Link to rule report
take_creates_coin_with_balance	Verified	<i>after take is called, a coin with a balance with the appropriate value is created</i>	rule report
put_increases_balance_by_coin_balance	Verified	<i>after put is called, the balance is increased by the value of the balance of the given coin</i>	rule report



join_increases_coin_value_by_coin_value	Verified	after join is called, the coin value is increased by the value of the given coin	rule report
split_decreases_coin_value_by_split_amount	Verified	after split is called, the coin value is decreased by the split amount	rule report
divide_into_n_does_not_increase_coin_value	Verified	after divide_into_n is called, the coin value is not increased	rule report
divide_into_n_creates_equal_value_coins	Verified	after divide_into_n is called, any two coins created have the same value	rule report
divide_into_n_does_not_change_sum_of_values	Verified	after divide_into_n is called, the sum of the values of the coins created plus self is equal to the original value of self	rule report
zero_creates_coin_with_value_zero	Verified	after zero is called, a coin with a value of zero is created	rule report
destroy_zero_deletes_coin	Verified	after destroy_zero is called, the coin is deleted	rule report
mint_creates_coin_with_value_equal_to_amount	Verified	after mint is called, a coin with a balance with the appropriate value is created	rule report
mint_balance_creates_balance	Verified	after mint_balance is called, a balance with the appropriate value is created	rule report
burn_deletes_coin	Verified	after burn is called, the coin is deleted	rule report



P-04. Initialization

Status: Verified

Currency creation (create_currency and create_regulated_currency_v1) establishes the correct initial state (zero supply and metadata).

Rule Name	Status	Description	Link to rule report
initialized_coin_has_zero_supply	Verified	<i>after create_currency is called, the supply of the coin is zero</i>	rule report
initialized_coin_has_right_decimals	Verified	<i>after create_currency is called, the decimals of the coin are the same as the input</i>	rule report
initialized_coin_has_right_name	Verified	<i>after create_currency is called, the name of the coin are the same as the input</i>	rule report
initialized_coin_has_right_description	Verified	<i>after create_currency is called, the description of the coin are the same as the input</i>	rule report
initialized_coin_has_right_icon_url	Verified	<i>after create_currency is called, the icon url of the coin are the same as the input</i>	rule report
initialized_regulated_coin_has_zero_supply	Verified	<i>after create_regulated_currency_v1 is called, the supply of the coin is zero</i>	rule report
initialized_regulated_coin_has_right_decimals	Verified	<i>after create_regulated_currency_v1 is called, the decimals of the coin are the same as the input</i>	rule report
initialized_regulated_coin_has_right_name	Verified	<i>after create_regulated_currency_v1 is called, the name of the coin are the same as the input</i>	rule report

initialized_regulate d_coin_has_right _description	Verified	<i>after create_regulated_currency_v1 is called, the description of the coin are the same as the input</i>	rule report
initialized_regulate d_coin_has_right _icon_url	Verified	<i>after create_regulated_currency_v1 is called, the icon url of the coin are the same as the input</i>	rule report
initialized_regulate d_coin_has_right _allow_global_pau se	Verified	<i>after create_regulated_currency_v1 is called, the allow_global_pause of the coin are the same as the input</i>	rule report

P-05. DenyList

Status: Verified	Deny-list and global-pause operations update state as specified (add/remove for next epoch, enable/disable global pause).
------------------	--

Rule Name	Status	Description	Link to rule report
deny_list_v1_add _adds_to_next_e poch	Verified	<i>after deny_list_v1_add is called, the given address is added to the deny list for the next epoch</i>	rule report
deny_list_v1_remove _removes_fro m_next_epoch	Verified	<i>after deny_list_v1_remove is called, the given address is removed from the deny list for the next epoch</i>	rule report
deny_list_v1_enable _global_pause_ enables_global_p ause	Verified	<i>after enable_global_pause is called, the global pause is enabled for the given coin type for the next epoch</i>	rule report
deny_list_v1_disable _global_pause	Verified	<i>after disable_global_pause is called, the global pause is disabled for the given coin type for the next epoch</i>	rule report



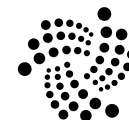
_disables_global_
pause

P-06. Metadata

Status: Verified

Metadata update functions (name, symbol, description, icon_url) set the corresponding fields as specified.

Rule Name	Status	Description	Link to rule report
update_name_integrity	Verified	<i>after update_name is called, the name of the coin is the given name</i>	rule report
update_symbol_integrity	Verified	<i>after update_symbol is called, the symbol of the coin is the given symbol</i>	rule report
update_description_integrity	Verified	<i>after update_description is called, the description of the coin is the given description</i>	rule report
update_icon_url_integrity	Verified	<i>after update_icon_url is called, the icon url of the coin is the given icon url</i>	rule report



Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading web3 security tools and services. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.